Browser Automation Tools: Approaches, Constraints & Recommendations

Oct 9, 2026 · @Shashank

Overview

Recommendation: build a server-side hybrid platform: deterministic login and cached scripts, with an LLM agent for the steps that vary. Use official APIs wherever they exist, especially for flight booking and payment. Add a browser extension later if an attended, in-browser mode is needed.

The requirement is tooling that operates websites on a user's behalf. Two reference tasks frame the analysis:

- **Form submission:** log in to a web application, fill out a specific form, submit it. Low risk and repeatable.
- **Flight booking:** search a booking site, pick the cheapest matching flight, book it. High risk: anti-bot defenses, changing UIs and real payments.

The document covers the server-side approaches, client-side options, the unattended ("offline") constraint, login and connection security, cross-cutting requirements, how Browserbase and TinyFish work, the wider vendor landscape, and recommendations.

Server-side approaches

These run in our own infrastructure, which makes them the only option that works for unattended tasks. Production systems usually combine several of them.

1. Scripted browser automation (Playwright, Puppeteer, Selenium)

Method: An engineer writes a script per site and workflow. It drives a real browser, finds elements with selectors (CSS, XPath, test IDs, ARIA roles), fills inputs and clicks. Each task becomes a callable function, for example `submitExpenseForm(credentials, data)`.

Required: Playwright (recommended for its auto-waiting, multi-browser support and tracing), a page-object model per site so selectors live in one place, containerized browsers, and ongoing script maintenance.

Pros:

- Fast and cheap per run, with no LLM calls.
- Deterministic, so the same input gives the same actions.
- Easy to debug with traces, screenshots and video.

Cons:

- Breaks when a site's UI changes.
- Every new site needs new engineering.
- Can't handle unexpected pop-ups or layouts.
- Plain headless browsers are often blocked by protected sites such as airlines.

2. API or network-level integration

Method: Skip the UI and call the system behind it. Use official APIs (for flights: Duffel, Amadeus, airline NDC APIs) or replay the network calls the site's frontend makes.

Required: API contracts or keys, auth flows (OAuth, session cookies, CSRF tokens), and for flights a commercial agreement and possibly ticketing accreditation.

Pros:

- The most reliable and fastest option, returning structured data.
- A legitimate, supported path for booking and payment.
- No browser infrastructure needed.

Cons:

- Not every site has an API.
- Private, reverse-engineered endpoints can break without warning or violate terms of service.
- Official travel APIs cost money and require onboarding.

3. LLM agent using the page structure (DOM / accessibility tree)

Method: An LLM receives the task in natural language. In a loop, it reads a compact snapshot of the page with numbered elements and chooses the next action, for example `click(e42)` or `type(e17, "Dallas")`. A Playwright executor performs the action and returns the new page state. Frameworks: Playwright MCP, Browser Use, Stagehand, or a custom tool-calling loop.

Required: An LLM with strong tool use, a browser tool layer (navigate, click, type, select, scroll, extract, wait), page-state compression, step budgets and stop conditions, structured outputs, and guardrails.

Pros:

- Works on sites nobody scripted.
- Recovers from pop-ups and layout changes.
- Handles fuzzy goals like "pick the cheapest nonstop."

Cons:

- Seconds per step and token cost on every step.
- Nondeterministic: it can take wrong paths or loop.
- Vulnerable to prompt injection hidden in page content.
- Needs evaluation before it handles anything consequential.

4. Vision-based computer-use agent

Method: The model sees screenshots and returns mouse and keyboard actions at screen coordinates, the way a person would. Examples: Claude computer use, OpenAI's computer-use model, Gemini computer use.

Required: A sandboxed VM or container with a display, a multimodal model, and screenshot and action plumbing.

Pros:

- Works where the page structure is unusable: canvas apps, legacy UIs, obfuscated markup.

Cons:

- The slowest and most expensive approach.
- Coordinate clicks are less precise.
- Screenshots can capture secrets and personal data.
- Best used only as a fallback.

5. Hybrid: learn once, replay deterministically (recommended core)

Method: The agent from approach 3 completes a workflow the first time, and the successful steps are saved as a cached script. Later runs replay the script with no LLM. When a step fails, the agent repairs it and updates the cache ("self-healing"). Stagehand's

action caching works this way.

Required: Everything from approaches 1 and 3, plus a versioned store of scripts per site and workflow, checks that each page reached its expected state, and human review before a new or repaired script becomes the default.

Pros:

- Scripted speed and cost on the normal path.
- Agent-level resilience when sites change.
- Reliability improves over time as repaired scripts accumulate.

Cons:

- The most moving parts to build.
- Repaired steps need review, or errors get cached and repeated.

6. RPA platforms (buy instead of build)

UiPath, Automation Anywhere and Microsoft Power Automate now include LLM-driven agents. They offer enterprise governance, audit trails, and both attended and unattended bots out of the box. The tradeoffs are license cost, lock-in and less developer flexibility.

Comparison of server-side approaches

The hybrid approach gives the best balance for a production platform. APIs win wherever they exist.

Criterion	Scripted	API-level	DOM agent	Vision agent	Hybrid
Reliability on known sites	High until the UI changes	Highest	Medium	Medium-low	High
Effort to add a new site	High	High, if an API exists	Low	Low	Low, then cached
Cost per run	Very low	Very low	Medium-high	High	Low on average
Speed	Fast	Fastest	Slow	Slowest	Fast
Handles UI changes	No	Not affected	Yes	Yes	Yes, self-heals

Criterion	Scripted	API-level	DOM agent	Vision agent	Hybrid
Exposure of secrets to the LLM	None	None	Avoidable with placeholders	High: screenshots	Avoidable
Best fit	Stable internal forms	Flights, payments	Long tail of sites	Canvas and legacy UIs	Production platform

Client-side approaches (running in the user's browser)

A browser extension is the main client-side option. An ordinary web page can't control other sites: the same-origin policy blocks reading or clicking in another site's tab or iframe, and most sites refuse to be framed. Client-side automation therefore needs an extension, or software installed on the user's machine.

1. Browser extension (main option)

Method: A Chrome/Edge extension (Manifest V3), usually with a side panel where the user starts tasks. It has three parts:

- **Content scripts** injected into the target site read the page and perform the fills, clicks and selections.
- **A background service worker** coordinates tabs and navigation and talks to our backend.
- **Our backend** runs the LLM agent loop. The extension sends page snapshots up and receives the next action. LLM calls and API keys stay on the server, never in the extension.

For sites that ignore events synthesized by JavaScript, the `chrome.debugger` API sends trusted mouse and keyboard input through the DevTools Protocol, at the cost of a visible "debugging this browser" banner. Claude in Chrome works this way.

Pros:

- Reuses the user's existing logged-in sessions, so we store no credentials and MFA happens naturally.
- Runs on the user's real browser and IP, so anti-bot defenses trigger far less.
- The user is present to approve payments or solve CAPTCHAs.
- No server-side browser infrastructure to run.

Cons:

- Requires installation: enterprise force-install policies, or Web Store review for consumers.
- Manifest V3 service workers can be suspended, so long tasks need resumable state.
- Runs only while the browser and tab are open, so it can't do unattended jobs.
- Powerful permissions make the extension an attack target. It needs narrow permissions, a domain allowlist and prompt-injection defenses.
- Harder to observe and debug, and users run mixed versions.

2. Local companion app controlling the user's browser

Method: A small desktop agent connects to the user's browser through the DevTools Protocol (for example Playwright's `connectOverCDP`) or pairs with a thin extension through Native Messaging.

Pros:

- Full Playwright capabilities with trusted input.
- Server-side scripts and the hybrid approach can be reused almost unchanged.

Cons:

- Heavier install with OS-level permissions.
- An exposed remote-debugging port is a security risk, and newer Chrome versions restrict it on the default profile.
- Support burden across Windows, macOS and Linux.

3. Desktop app with an embedded browser (Electron, Tauri, WebView2)

Method: Our app ships its own Chromium. Users log in to sites inside the app, which has full programmatic control.

Pros:

- Complete control and a predictable environment.
- No extension store rules; good for internal tools where we control distribution.

Cons:

- Doesn't share the user's normal browser sessions.
- Anti-bot systems sometimes flag embedded browsers.
- A large app to build, update and secure.

4. Userscripts and bookmarklets

JavaScript run on one site through Tampermonkey or a bookmarklet. Trivial to build and fine for a prototype. Limited to one page at a time, fragile, hard to distribute, and not suitable for a product.

Combining client and server

Client and server executors can share one agent backend, one set of task definitions and one action vocabulary (navigate, click, type, extract). Only the executor differs:

- **Client extension:** tasks that need the user's identity, MFA or approval. Example: searching flights and confirming payment in the user's own session.
- **Server-side browsers:** unattended, scheduled or high-volume jobs using vault-stored credentials. Example: nightly form submissions across many accounts.

Constraint: unattended (offline) execution

If tasks must run while the user is absent and not signed in, only server-side execution is reliable. Every server-side approach works from a scheduler or job queue in disposable browser containers, with traces stored for later review.

Option	Feasible unattended?	Why
Server-side approaches 1–6	Yes	Run entirely in our infrastructure from a queue
Browser extension	No	Needs the user's browser open and running
Userscripts, bookmarklets	No	Need the user's browser and a click
Local companion app	Partially	Only while the machine is on, awake and online; not dependable
Embedded-browser desktop app	Partially	Same limitation as the companion app

Authenticating without the user

Without a live user session, the server must log in on the user's behalf. Options in order of preference:

1. **Official APIs with delegated authorization:** OAuth refresh tokens or partner APIs. The user grants access once, and we never hold a password.
2. **Credentials in a secrets vault:** encrypted, scoped per user, auditable and revocable. Only the executor injects them at runtime; they never appear in prompts.
3. **Saved sessions:** the user logs in once during a supervised session, and the cookies and storage are saved for reuse. Sessions expire, so a fallback is still needed.
4. **Service accounts:** a dedicated automation account for internal or partner apps, which avoids impersonating real users.

MFA is the hardest blocker:

- Authenticator-app (TOTP) codes can be automated by storing the TOTP secret in the vault, where policy and the site's terms allow.
- Email codes can sometimes be read through delegated mailbox access.
- SMS codes and push approvals generally can't be automated. The job should pause and notify the user rather than fail silently.

Approvals and payments while the user is away

- **Asynchronous approval:** the job finds the cheapest option and stages or holds the booking. It sends an approval link by email, Slack or push, and books only after approval.
- **Pre-authorized rules:** the user approves rules in advance, for example "nonstop, under $400, departing before 10 AM." The job pays with a tokenized card or a single-use virtual card capped at that amount, and card data never reaches the LLM.

For flights, unattended runs should use a flight API rather than airline websites. Headless browsers on airline sites face the heaviest anti-bot defenses, CAPTCHAs no one is present to solve, and terms-of-service restrictions.

Login handling and secure connections

All approaches use the same login building blocks; they differ in where credentials live and who can see them. The rule that matters most: the LLM never sees a real secret.

Login methods

- **Token-based access:** OAuth, API keys or service accounts. No password is handled, and tokens are scoped and revocable. The safest option.
- **Vault-injected credentials:** secrets stay in GCP Secret Manager, HashiCorp Vault or similar. The executor fetches one just before the login step and types it directly into the field. Mask password fields in logs, traces and recordings.
- **Saved sessions:** log in once and reuse the cookies and storage (Playwright storage state, Browserbase or Steel contexts). A saved session is a credential in its own right, so encrypt it, keep it per user, rotate it, and detect expiry.
- **Human handoff:** for SMS codes, push approvals, CAPTCHAs or new-device checks, pause the job. The user completes the step through a live view of the remote browser, or by logging in once so a fresh session can be saved.

How each approach handles login

Approach	Login handling	Main risk to control
Scripted	Code pulls the secret from the vault, fills it, saves the session	Secrets in logs or traces; unencrypted session files
API-level	OAuth tokens or API keys	Token storage and scope
DOM agent	Placeholders (e.g. `%password%` in Stagehand, Browser Use's sensitive-data option); the executor substitutes real values	Secrets in page snapshots sent to the model
Vision agent	Script the login, then hand the logged-in session to the agent	Screenshots capturing secrets
Hybrid	Deterministic login script; the agent works only after login	Cleanest split; recommended
Browser extension	Uses the user's existing session; nothing stored	Extension permissions, personal data sent to the backend, prompt injection
Managed vendors	The vendor's contexts and credential features	Third-party trust: encryption, recordings, SOC 2, data location, deletion

Securing the connections

- **Browser to website:** standard HTTPS. Never disable certificate checks. Use only trusted proxy providers, because a proxy sits in the network path.
- **Our system to the remote browser:** the DevTools Protocol connection gives full control of the browser. Use encrypted WebSockets (`wss://`) with token authentication and short-lived session URLs. When self-hosting, keep the debugging port on a private network, never public.
- **Our system to the LLM:** page content leaves our systems. Redact sensitive fields, use zero-data-retention or enterprise terms, and consider a regionally hosted model for sensitive data.
- **Isolation and audit:** one disposable container per task, with a fresh or per-user profile, destroyed afterwards. Restrict outbound traffic to an allowlist of domains per task. Log every secret access and every action, with masked screenshots.

Requirements for any approach

Whichever approach is chosen, the platform needs these building blocks.

Area	What's needed
Browser infrastructure	Isolated, disposable sessions per task: self-hosted containerized Chromium (e.g. on GKE with autoscaling) or a hosted provider (Browserbase, Steel, Browserless). Persistent per-tenant sessions with no cross-tenant leakage.
Credentials	Secrets manager with runtime injection; TOTP support where allowed; human-in-the-loop for SMS and push MFA.
Approval gates	Explicit human approval, or pre-authorized rules, for any irreversible or financial action. Payments through PCI-compliant tokenized or virtual cards, never the LLM.
Agent guardrails	Domain allowlist per task, allowed-action list, step and cost budgets, page content treated as untrusted data so it can't change the task.

Area	What's needed
Anti-bot and legal	Check each site's terms of service. Prefer official APIs for travel. Route CAPTCHAs to a human or an API rather than automating them on third-party sites.
Observability	Step-by-step traces with before/after screenshots, Playwright traces or video, agent reasoning per step, structured results.
Orchestration	Task schema (inputs, success criteria, outputs, approval policy); job queue with retries, timeouts, status callbacks, and idempotency keys so a retry never books twice.
Evaluation	Regression suite of real workflows on staging or sandbox sites. Track success rate, steps, cost and latency per site; run on every model or prompt change and on a schedule.

How Browserbase and TinyFish work

Browserbase mainly sells the browser infrastructure plus an open-source agent framework; TinyFish sells the whole agent as an API. Both follow the architecture recommended here: server-side execution, persisted sessions, agents that read page structure, and deterministic replay where possible. Neither publishes full internals; this is based on their docs and public material.

Browserbase

- **Cloud browsers:** managed headless Chromium, controlled over the DevTools Protocol, so existing Playwright or Puppeteer code connects with little change.
- **Session features:** persistent contexts keep logins across sessions; proxies, Advanced Stealth and long-running sessions are available (Stagehand MCP docs). Running on Browserbase gives advanced fingerprinting versus basic stealth locally (Stagehand browser docs).
- **Stagehand:** an open-source SDK that mixes Playwright code with AI primitives (`act`, `extract`, `observe`, `agent`). It caches resolved actions so repeat runs skip the LLM, which is the hybrid approach.

- **Managed agents:** Browserbase Agents takes a goal through an API and returns structured results and recordings; Search and Fetch cover cases where a full browser isn't needed (AI Engineer profile).

TinyFish

- **One API call:** describe the goal and target site, and a managed agent returns structured results (overview).
- **Page structure, not screenshots:** its agents act on the underlying structure of pages rather than visual interpretation (TinyFish blog), the DOM-agent approach.
- **Production scale:** described as distributed agents that log in, handle authentication, get past anti-bot systems and adapt to page changes (Unite.AI).
- **Layered tools:** search and page fetch alongside full agents, with the guidance to use search first, fetch second and agents last; an MCP server lets MCP-aware AI clients call it (Bitdoze).
- **Pricing and claims:** per-step pricing with concurrency limits, from $15/month (1,650 steps, 10 concurrent agents). A cited benchmark of 81% task success is vendor-favorable and should be verified on our own workflows.

What we would still own with either vendor

Layer	Build ourselves	Browserbase	TinyFish
Browser runtime, scaling, stealth, proxies	Us	Provided	Provided
Session persistence and recordings	Us	Provided	Provided
Agent loop, element finding, caching	Us	Stagehand (open source)	Provided
Task logic, guardrails, approvals, credentials	Us	Us	Mostly us
Control and customization	Full	High	Lower

Vendor landscape

Vendors fall into five layers, from raw browsers we build on to fully managed agents. Several "best alternatives" lists are published by competitors, so treat rankings with care.

Layer	Options	Notes
Cloud browser infrastructure	Browserbase, Browserless, Steel, Hyperbrowser, Bright Data Scraping Browser	Browsers only; we build the logic. Browserless and Steel can be self-hosted. Bright Data is strongest on proxies for protected sites.
Open-source agent frameworks	Stagehand, Browser Use, Skyvern, Playwright MCP	We host and control them. Skyvern targets form-heavy and login-heavy workflows; Browser Use can also drive the user's real browser.
Managed agent platforms	TinyFish, Browserbase Agents, MultiOn, Skyvern Cloud, Browser Use Cloud, Apify	Goal in, result out. Apify is stronger for data extraction than for transactions.
Model-level computer use	Claude computer use and Claude in Chrome, OpenAI computer use and ChatGPT agent, Gemini computer use and Project Mariner	API versions can be the brain of our own system.
RPA with AI agents	UiPath, Automation Anywhere, Power Automate	Best if already in use or if enterprise governance is needed immediately.

Shortlist

- **Build with full control:** Stagehand or Browser Use on self-hosted Browserless or Steel in our own cloud. Data stays in our infrastructure; most engineering effort.
- **Build faster:** Stagehand on Browserbase, or Skyvern Cloud for form-heavy workflows. Managed stealth and sessions with our logic on top.
- **Buy:** TinyFish or Browserbase Agents. Least engineering, least control.

Recommendations and next steps

Build server-side first, on the hybrid approach, with official APIs for anything involving payment.

By task

- **Log in, fill form, submit:** deterministic Playwright login with vault-injected credentials and an encrypted per-user saved session. Add the self-healing hybrid layer once there are more than a handful of sites. If the app is ours, call its API instead.
- **Flight search and booking:** use a flight API (Duffel, Amadeus or similar) for search, booking and payment, with asynchronous approval or pre-authorized rules and a tokenized or virtual card. Use browser agents only where no API exists.

Target architecture

1. A task API and job queue receive tasks defined by a schema.
2. The orchestrator picks an execution path: a cached script if one exists, otherwise the DOM agent, with the vision agent as a last resort.
3. A Playwright executor runs in disposable, sandboxed browser containers, self-hosted or from a hosted provider.
4. Supporting services: secrets vault, approval service, and a trace and recording store.
5. Optional later: a browser extension executor for attended tasks, sharing the same backend and task definitions.

Next steps

- [ ] Confirm whether the target sites have official APIs, starting with the flight-booking use case.
- [ ] Run a proof of concept of both reference tasks on Stagehand with Browserbase, and on TinyFish.
- [ ] Compare success rate, cost per task, and login and MFA handling across the proof-of-concept runs.
- [ ] Decide build versus buy, and self-hosted versus hosted browsers, based on data-residency needs.
- [ ] Review terms of service for each target site with legal.
- [ ] Define the task schema, approval policy and guardrails before the first production workflow.